

有限元期末大作业求解报告

1 问题分析

给定偏微分方程边值问题：

$$\begin{cases} -\Delta u + 4u = f, & \mathbf{x} \in \Omega \\ u = g, & \mathbf{x} \in \partial\Omega \end{cases}$$

其中：

- Ω 为梯形区域 (如图1)
- $f(\mathbf{x}) = e^{2x+2y}(4 \sin(2x+2y) - 16 \cos(2x+2y))$
- $g(\mathbf{x}) = e^{2x+2y} \sin(2x+2y)$
- 精确解 $u_{exact}(\mathbf{x}) = e^{2x+2y} \sin(2x+2y)$

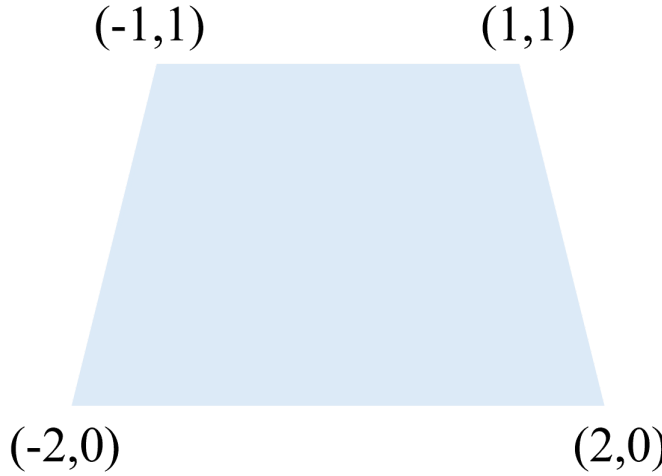


图1 梯形区域

1.1 方程验证

验证精确解满足原方程：

$$\begin{aligned} u &= e^{2x+2y} \sin(2x+2y) \\ \nabla u &= e^{2x+2y} \begin{bmatrix} 2 \sin(2x+2y) + 2 \cos(2x+2y) \\ 2 \sin(2x+2y) + 2 \cos(2x+2y) \end{bmatrix} \\ \Delta u &= \nabla \cdot (\nabla u) = 8e^{2x+2y} \cos(2x+2y) \\ -\Delta u + 4u &= -8e^{2x+2y} \cos(2x+2y) + 4e^{2x+2y} \sin(2x+2y) \\ &= e^{2x+2y}(4 \sin(2x+2y) - 8 \cos(2x+2y)) \\ &= f(\mathbf{x}) \quad \text{得证} \end{aligned}$$

2 有限元方法推导

2.1 变分形式

将方程转化为弱形式。乘以测试函数 $v \in H_0^1(\Omega)$ 并积分：

$$\int_{\Omega} (-\Delta u + 4u) v d\Omega = \int_{\Omega} f v d\Omega$$

应用Green公式：

$$\int_{\Omega} \nabla u \cdot \nabla v d\Omega + 4 \int_{\Omega} u v d\Omega - \int_{\partial\Omega} \frac{\partial u}{\partial n} v ds = \int_{\Omega} f v d\Omega$$

考虑Dirichlet边界条件 $u|_{\partial\Omega} = g$, 得变分形式:

$$a(u, v) = f(v) \quad \forall v \in H_0^1(\Omega)$$

其中:

$$a(u, v) = \int_{\Omega} \nabla u \cdot \nabla v d\Omega + 4 \int_{\Omega} u v d\Omega$$

$$f(v) = \int_{\Omega} f v d\Omega$$

2.2 六节点三角形单元

使用二次Lagrange基函数:

- 每个三角形单元6个节点 (3顶点 + 3边中点)
- 单元基函数满足 $\varphi_i(\mathbf{x}_j) = \delta_{ij}$

参考单元($\hat{\tau}$)基函数 (标准三角元) :

$$N_1 = (1 - \xi - \eta)(1 - 2\xi - 2\eta)$$

$$N_2 = \xi(2\xi - 1)$$

$$N_3 = \eta(2\eta - 1)$$

$$N_4 = 4\xi(1 - \xi - \eta)$$

$$N_5 = 4\xi\eta$$

$$N_6 = 4\eta(1 - \xi - \eta)$$

2.3 单元矩阵计算

设实际单元为 T , 通过仿射映射 $F_T: \hat{\tau} \rightarrow T$

$$\mathbf{x} = F_T(\hat{\mathbf{x}}) = \sum_{i=1}^3 \mathbf{x}_i \phi_i(\hat{\mathbf{x}})$$

刚度矩阵元素:

$$\begin{aligned} a_{ij}^T &= \int_T \nabla \varphi_i \cdot \nabla \varphi_j dxdy + 4 \int_T \varphi_i \varphi_j dxdy \\ &= |J_T| \int_{\hat{\tau}} (J_T^{-T} \nabla N_i) \cdot (J_T^{-T} \nabla N_j) d\xi d\eta \\ &\quad + 4|J_T| \int_{\hat{\tau}} N_i N_j d\xi d\eta \end{aligned}$$

右端向量元素:

$$f_i^T = \int_T f \varphi_i dxdy = |J_T| \int_{\hat{\tau}} f(F_T(\hat{\mathbf{x}})) N_i d\xi d\eta$$

使用五阶7点高斯积分计算:

表1 五阶7点高斯积分

i	ξ_i	η_i	w_i
1	$\frac{1}{3}$	$\frac{1}{3}$	$\frac{9}{80}$
2	$\frac{6+\sqrt{15}}{21}$	$\frac{6+\sqrt{15}}{21}$	$\frac{155+\sqrt{15}}{2400}$
3	$\frac{9-2\sqrt{15}}{21}$	$\frac{6+\sqrt{15}}{21}$	$\frac{155+\sqrt{15}}{2400}$
4	$\frac{6+\sqrt{15}}{21}$	$\frac{9-2\sqrt{15}}{21}$	$\frac{155+\sqrt{15}}{2400}$
5	$\frac{6-\sqrt{15}}{21}$	$\frac{6-\sqrt{15}}{21}$	$\frac{155-\sqrt{15}}{2400}$
6	$\frac{9+2\sqrt{15}}{21}$	$\frac{6-\sqrt{15}}{21}$	$\frac{155-\sqrt{15}}{2400}$
7	$\frac{6-\sqrt{15}}{21}$	$\frac{9+2\sqrt{15}}{21}$	$\frac{155-\sqrt{15}}{2400}$

3 误差计算

采用 L^2 范数误差估计：

$$\|u - u_h\|_{L^2(\Omega)} = (\int_{\Omega} (u - u_h)^2 d\Omega)^{1/2}$$

使用单元积分：

$$\|u - u_h\|_{L^2(\Omega)}^2 = \sum_T |J_T| \int_{\hat{\tau}} [u(F_T(\hat{\mathbf{x}})) - u_h(F_T(\hat{\mathbf{x}}))]^2 d\xi d\eta$$

4 结果分析

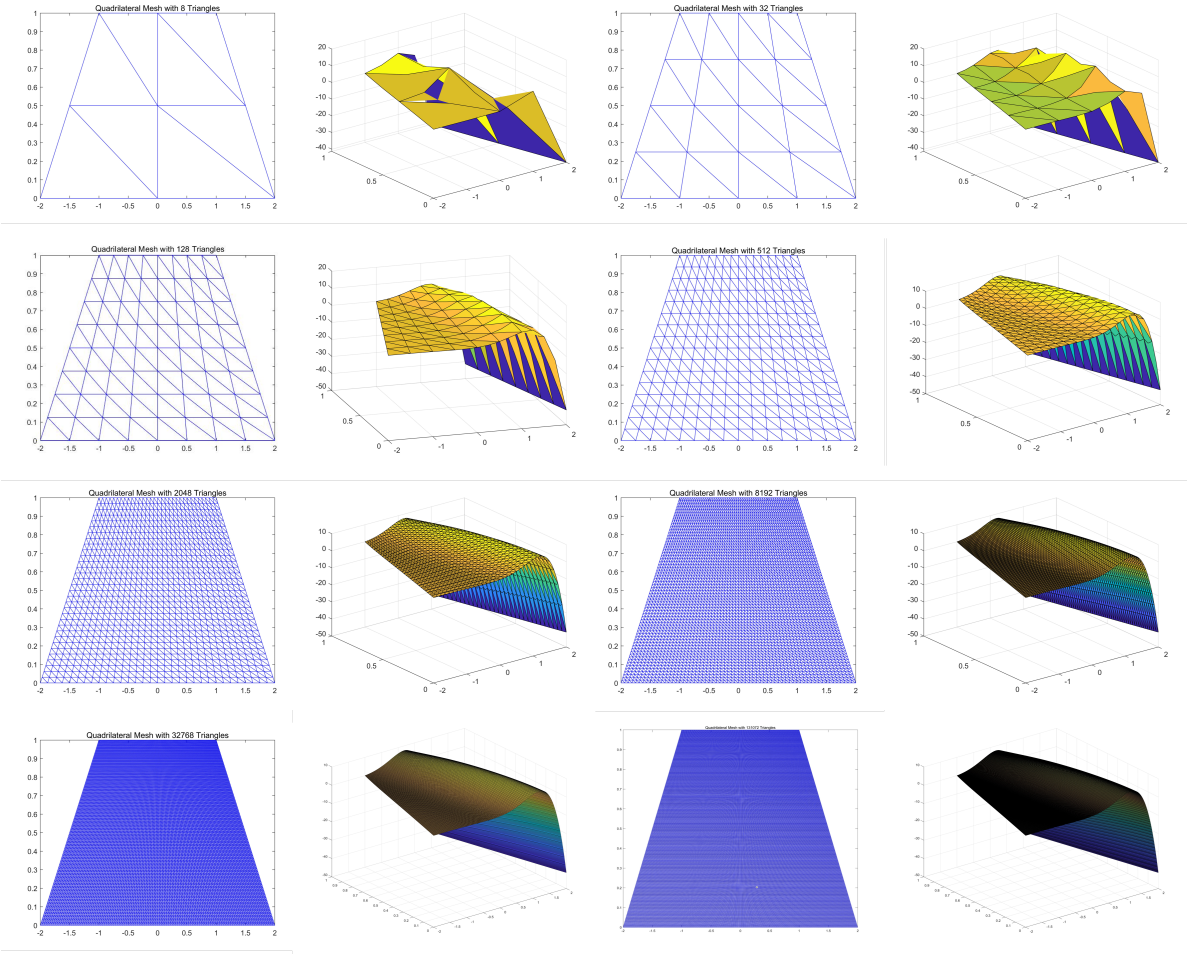


图2 求解结果可视化

如上图所示，当网格密度划分层数从1增加到8时，结果越来越精准，符合结果预期。

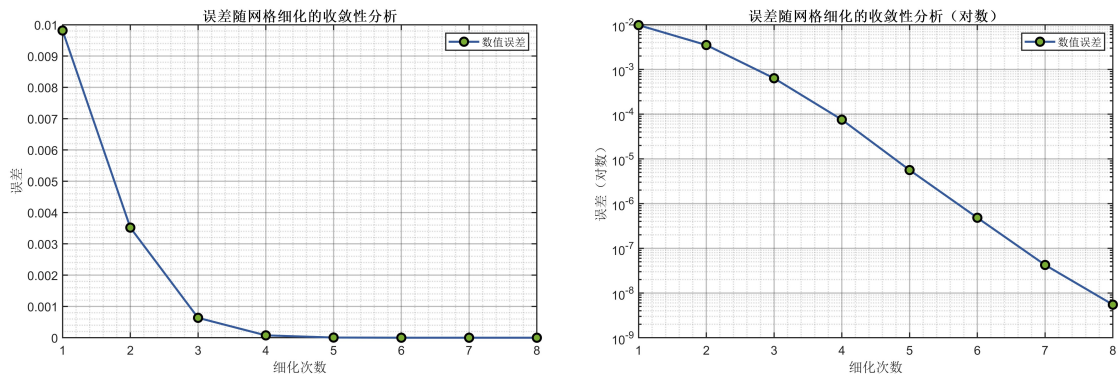


图3 求解结果可视化

如上图所示，当网格密度划分层数从1增加到8时，相对L2误差越来越小，符合结果预期。

5 代码

trapezoid2d(k)

```
function trapezoid2d(k)
% 四边形顶点定义
P1 = [-2, 0]; % 左下
P2 = [ 2, 0]; % 右下
P3 = [ 1, 1]; % 右上
P4 = [-1, 1]; % 左上

m = 2^k; % 每边分割数
n_xi = m + 1; % xi方向节点数
n_eta = m + 1; % eta方向节点数

% 生成参数域网格
xi = linspace(0, 1, n_xi);
eta = linspace(0, 1, n_eta);

% 坐标生成(双线性插值)
np = n_xi * n_eta;
X = zeros(np,1);
Y = zeros(np,1);
xi_param = zeros(np,1);
eta_param = zeros(np,1);

idx = 1;
for j = 1:n_eta
    for i = 1:n_xi
        xi_i = xi(i);
        eta_j = eta(j);

        % 双线性插值
        X(idx) = (1-xi_i)*(1-eta_j)*P1(1) + xi_i*(1-eta_j)*P2(1) + ...
                xi_i*eta_j*P3(1) + (1-xi_i)*eta_j*P4(1);
        Y(idx) = (1-xi_i)*(1-eta_j)*P1(2) + xi_i*(1-eta_j)*P2(2) + ...
                xi_i*eta_j*P3(2) + (1-xi_i)*eta_j*P4(2);

        % 存储参数坐标
        xi_param(idx) = xi_i;
```

```

        eta_param(idx) = eta_j;

        idx = idx + 1;
    end
end

% 生成三角形连接性
triangles = zeros(2*m*m, 3);
tri_idx = 1;
for j = 1:m
    for i = 1:m
        ll = (j-1)*n_xi + i;
        lr = ll + 1;
        ul = ll + n_xi;
        ur = ul + 1;

        triangles(tri_idx,:) = [ll, lr, ul];
        triangles(tri_idx+1,:) = [lr, ur, ul];
        tri_idx = tri_idx + 2;
    end
end

% 创建三角剖分对象
t = triangulation(triangles, X, Y);

% 获取所有边界边
edges = t.edges();
tol = 1e-10;

% 生成唯一边并计算中点
unique_edges = containers.Map('KeyType','char','ValueType','any');
for edge_idx = 1:size(edges, 1)
    p1 = edges(edge_idx, 1);
    p2 = edges(edge_idx, 2);
    if p1 > p2
        [p1, p2] = deal(p2, p1);
    end
    key = sprintf('%d-%d', p1, p2);
    if ~iskey(unique_edges, key)
        unique_edges(key) = struct('p1', p1, 'p2', p2);
    end
end

% 计算中点并扩展顶点列表
num_edges = size(edges, 1);
X_new = [X; zeros(num_edges, 1)];
Y_new = [Y; zeros(num_edges, 1)];
xi_new = [xi_param; zeros(num_edges, 1)];
eta_new = [eta_param; zeros(num_edges, 1)];
midpoint_map = containers.Map('KeyType','char','ValueType','double');

current_midpoint_index = np + 1;
for key = unique_edges.keys()
    key_str = key{1};
    data = unique_edges(key_str);
    p1 = data.p1;
    p2 = data.p2;

```

```

% 计算中点坐标
mid_x = (X(p1) + X(p2)) / 2;
mid_y = (Y(p1) + Y(p2)) / 2;

% 计算中点参数坐标
mid_xi = (xi_param(p1) + xi_param(p2)) / 2;
mid_eta = (eta_param(p1) + eta_param(p2)) / 2;

% 存储中点
midpoint_map(key_str) = current_midpoint_index;
X_new(current_midpoint_index) = mid_x;
Y_new(current_midpoint_index) = mid_y;
xi_new(current_midpoint_index) = mid_xi;
eta_new(current_midpoint_index) = mid_eta;

current_midpoint_index = current_midpoint_index + 1;
end

% 构建六节点三角形单元
triangles_new = zeros(size(triangles, 1), 6);
for tri_idx = 1:size(triangles, 1)
    v1 = triangles(tri_idx, 1);
    v2 = triangles(tri_idx, 2);
    v3 = triangles(tri_idx, 3);

    % 获取各边中点
    m12 = midpoint_map(sprintf('%d-%d', min(v1,v2), max(v1,v2)));
    m23 = midpoint_map(sprintf('%d-%d', min(v2,v3), max(v2,v3)));
    m31 = midpoint_map(sprintf('%d-%d', min(v3,v1), max(v3,v1)));

    % 按逆时针顺序排列
    triangles_new(tri_idx, :) = [v1, m12, v2, m23, v3, m31];
end

% 确定边界节点
boundaryNodes = find((abs(xi_new) < tol) | (abs(xi_new - 1) < tol) | ...
    (abs(eta_new) < tol) | (abs(eta_new - 1) < tol));

figure;
triplot(t);
title(sprintf('Quadrilateral Mesh with %d Triangles', size(triangles,1)));

% 保存文件
fid = fopen('coordinates.dat', 'w');
fprintf(fid, '%24.16e %24.16e\n', [X_new, Y_new]');
fclose(fid);

fid = fopen('elements6.dat', 'w');
fprintf(fid, '%6d %6d %6d %6d %6d %6d\n', triangles_new');
fclose(fid);

fid = fopen('dirichletNodes.dat', 'w');
fprintf(fid, '%6d\n', boundaryNodes);
fclose(fid);

```

```

fprintf('细化级别 %d: 节点数 = %d, 单元数 = %d\n', k, length(X_new),
size(triangles_new,1));
end

```

[coordinates,u]=fem2dlinear()

```

function [coordinates,u]=fem2dlinear
% 加载网格数据
coordinates = load('coordinates.dat');
elements6 = load('elements6.dat');
dirichletNodes = load('dirichletNodes.dat'); % 边界节点

% 总节点数
nNodes = size(coordinates, 1);

% 初始化全局刚度矩阵和载荷向量
A = sparse(nNodes, nNodes);
b = sparse(nNodes, 1);

% 遍历所有单元
nElements = size(elements6, 1);
for e1 = 1:nElements
    % 获取单元节点
    nodes = elements6(e1, :);
    verts = coordinates(nodes, :);

    % 计算单元刚度矩阵和载荷向量
    [Ke, Fe] = elemStiffness(verts);

    % 组装到全局矩阵
    A(nodes, nodes) = A(nodes, nodes) + Ke;
    b(nodes) = b(nodes) + Fe;
end

u = zeros(nNodes, 1);
u_exact = exact_solution(coordinates);
u(dirichletNodes) = u_exact(dirichletNodes); % 直接设置边界值

% 分离自由节点和边界节点
freeNodes = setdiff(1:nNodes, dirichletNodes);

% 修正右端项
b(freeNodes) = b(freeNodes) - A(freeNodes, dirichletNodes) * u(dirichletNodes);

% 求解自由节点上的解
u(freeNodes) = A(freeNodes, freeNodes) \ b(freeNodes);

% % Graphic representation.
figure();
trisurf ( elements6, coordinates(:,1), coordinates(:,2), full(u) );
end

```

[N, dN] = shapeFunctions(L1, L2, L3)

```
function [N, dN] = shapeFunctions(L1, L2, L3)
    N = [L1*(2*L1-1);
         L2*(2*L2-1);
         L3*(2*L3-1);
         4*L1*L2;
         4*L2*L3;
         4*L3*L1];

    % 形函数导数 (对L1和L2)
    dN = zeros(6,2);
    dN(1,:) = [4*L1-1, 0];           % dN1/dL1, dN1/dL2
    dN(2,:) = [0, 4*L2-1];           % dN2/dL1, dN2/dL2
    dN(3,:) = [0, 0];                 % dN3/dL1, dN3/dL2 (将用链式法则处理)
    dN(4,:) = [4*L2, 4*L1];           % dN4/dL1, dN4/dL2
    dN(5,:) = [-4*L2, 4*(1-2*L2-L1)]; % dN5/dL1, dN5/dL2
    dN(6,:) = [4*(1-2*L1-L2), -4*L1]; % dN6/dL1, dN6/dL2

    % 处理dN3 (使用链式法则)
    dN3_dL3 = 4*L3 - 1;
    dN(3,:) = [-dN3_dL3, -dN3_dL3]; % dL3/dL1 = -1, dL3/dL2 = -1
end
```

val = f(points)

```
function val = f(points)
    x = points(:,1); y = points(:,2);
    val = exp(2*x+2*y) .* (4*sin(2*x+2*y) - 16*cos(2*x+2*y));
end
```

val = exact_solution(points)

```
function val = exact_solution(points)
    x = points(:,1); y = points(:,2);
    val = exp(2*x+2*y) .* sin(2*x+2*y);
end
```

[Ke, Fe] = elemStiffness(vertices)

```
function [Ke, Fe] = elemStiffness(vertices)
    % 高斯积分点 (面积坐标) - 7点积分
    gauss_points = [
        1/3, 1/3, 1/3;
        0.0597158717, 0.4701420641, 0.4701420641;
        0.4701420641, 0.0597158717, 0.4701420641;
        0.4701420641, 0.4701420641, 0.0597158717;
        0.7974269853, 0.1012865073, 0.1012865073;
        0.1012865073, 0.7974269853, 0.1012865073;
        0.1012865073, 0.1012865073, 0.7974269853
    ];

    weights = [
        0.225;
    ];
```



```

0.1323941527;
0.1323941527;
0.1323941527;
0.1259391805;
0.1259391805;
0.1259391805
];

Ke = zeros(6,6);
Fe = zeros(6,1);

for i = 1:7
    L1 = gauss_points(i, 1);
    L2 = gauss_points(i, 2);
    L3 = gauss_points(i, 3);

    % 计算形函数和导数
    [N, dN] = shapeFunctions(L1, L2, L3);

    % 计算雅可比矩阵 (修正为2x2)
    dxdL = [dN(:,1)'*vertices(:,1), dN(:,1)'*vertices(:,2);
            dN(:,2)'*vertices(:,1), dN(:,2)'*vertices(:,2)];

    J = dxdL; % 2x2雅可比矩阵
    detJ = abs(det(J));

    % ===== 关键修改3: 避免奇异矩阵 =====
    if detJ < 1e-14
        error('雅可比矩阵奇异, 单元退化');
    end

    invJ = inv(J);

    % 转换导数到物理坐标系
    dNdx = zeros(6,2);
    for j = 1:6
        dNd1 = [dN(j,1); dN(j,2)];
        dNdx(j,:) = (invJ * dNd1)';
    end

    % 计算积分点物理坐标
    x_phys = N' * vertices(:,1);
    y_phys = N' * vertices(:,2);

    % 计算Ke项
    for m = 1:6
        for n = 1:6
            Ke(m,n) = Ke(m,n) + (dNdx(m,:)*dNdx(n,:) + 4*N(m)*N(n)) * detJ
* weights(i);
        end
        % 计算Fe项
        Fe(m) = Fe(m) + f([x_phys, y_phys]) * N(m) * detJ * weights(i);
    end
end
end
end

```

[err] = error_calculate(u, u_exact)

```
function [err] = error_calculate(u, u_exact)
coordinates = load('coordinates.dat');
dirichletNodes = load('dirichletNodes.dat'); % 边界节点
nNodes = size(coordinates, 1);

err_nodes = setdiff(1:nNodes, dirichletNodes);
num_free = length(err_nodes);

err_numerator = 0;
err_denominator = 0;

for i = 1:num_free
    node_idx = err_nodes(i);
    err_numerator = err_numerator + (u(node_idx) - u_exact(node_idx))^2;
    err_denominator = err_denominator + u_exact(node_idx)^2;
end

err = sqrt(err_numerator) / sqrt(err_denominator);
end
```

main.m

```
clc;
clear;

% 初始化误差数组
N = 8; % 细化次数
errors = zeros(N, 1);

for k = 1:N
    fprintf('正在处理第 %d/%d 层细化...\n', k, N);
    % 生成网格
    trapezoid2d(k);
    % 计算数值解
    [coordinates,u]=fem2dlinear();
    % 计算精确解
    u_exact = exact_solution(coordinates);
    errors(k) = error_calculate(u, u_exact);
end

% 绘制误差曲线
figure('Position', [100, 100, 800, 500]); % 设置图形窗口位置和大小

% 自定义颜色和标记样式
h = plot(1:N, errors, 'o-', ...
    'Linewidth', 2, ...
    'MarkerSize', 8, ...
    'MarkerFaceColor', [0.4660, 0.6740, 0.1880], ... % 绿色填充
    'Color', [0.2000, 0.3500, 0.6100], ... % 深蓝色线条
    'MarkerEdgeColor', 'k'); % 黑色标记边框

% 设置坐标轴
grid on;
```

```
grid minor; % 添加次要网格线
set(gca, ...
    'Linewidth', 1.2, ...
    'FontSize', 12, ...
    'GridAlpha', 0.3, ...
    'MinorGridAlpha', 0.15);

ylabel('相对L2误差');

% 设置x轴
xlabel('细化次数');
xticks(1:N); % 设置x轴刻度为整数

% 添加标题和图例
title('误差随网格细化的收敛性分析', 'FontSize', 14, 'Fontweight', 'bold');
legend('数值误差', 'Location', 'northeast');
```